Assignment 2: Process Management

Operating System Lab (Unix)

Problem Statement

Write C programs to demonstrate process creation, execution, and synchronization in a Unix/Linux environment using system calls. The objective is to understand process lifecycle, parent-child relationships, and process control mechanisms.

Tasks to Implement

1. Write a program using `fork()` to create a child process and display:
   - Process ID (PID)
   - Parent Process ID (PPID)
2. Modify the above program to demonstrate:
   - Parent and child executing different code sections
3. Write a program using `wait()` to ensure parent waits for child process to finish
4. Write a program to demonstrate `exec()` family of functions (e.g., `execl()`, `execvp()`)
5. Write a program to create multiple child processes using `fork()` inside a loop
6. Write a program to demonstrate Zombie Process
7. Write a program to demonstrate Orphan Process

Instructions

- Use C programming language with system calls
- Compile using `gcc filename.c -o output`
- Execute using `./output`
- Use appropriate headers: `unistd.h`, `sys/types.h`, `sys/wait.h`
- Add comments to explain process behavior
- Use `sleep()` where necessary to observe process states

Sample Output

Example (fork):

```
Parent Process: PID = 1024
Child Process: PID = 1025, PPID = 1024
```

Example (wait):

```
Child process executing...
Child process completed.
Parent resumes execution.
```

Constraints

- Use only system calls (no high-level abstractions)
- Programs must compile and execute without errors
- Clearly demonstrate process behavior
- Avoid infinite loops unless required for demonstration

Evaluation Criteria

- Correctness of Output – 40%
- Understanding of Process Concepts – 20%
- Code Structure and Clarity – 20%
- Use of System Calls – 10%
- Documentation / Comments – 10%

Submission Guidelines

- Submit all programs in a single compressed (.zip) file
- Folder name: `RollNo_Assignment2`
- Include source files and compiled outputs
- Include a README explaining each program

Bonus Tasks (Optional but Recommended)

1. Write a program to create a process tree (parent → child → grandchild)
2. Use `exec()` to run Linux commands (e.g., `ls`, `date`)
3. Write a program to measure execution time of a child process
4. Demonstrate difference between `fork()` and `vfork()`
5. Implement a mini shell that creates a process to execute user commands